

Task 2: Conceptual Model

Airbnb-Style Accommodation Booking System

GROUP 89

GROUP MEMBERS:

Arshit – 2024112

Anshuman Nayak – 2024090

Ahin Raj S R – 2024048

TUTORIAL GROUP 7

TAs : Vishal Jha & Gauranshi Gupta

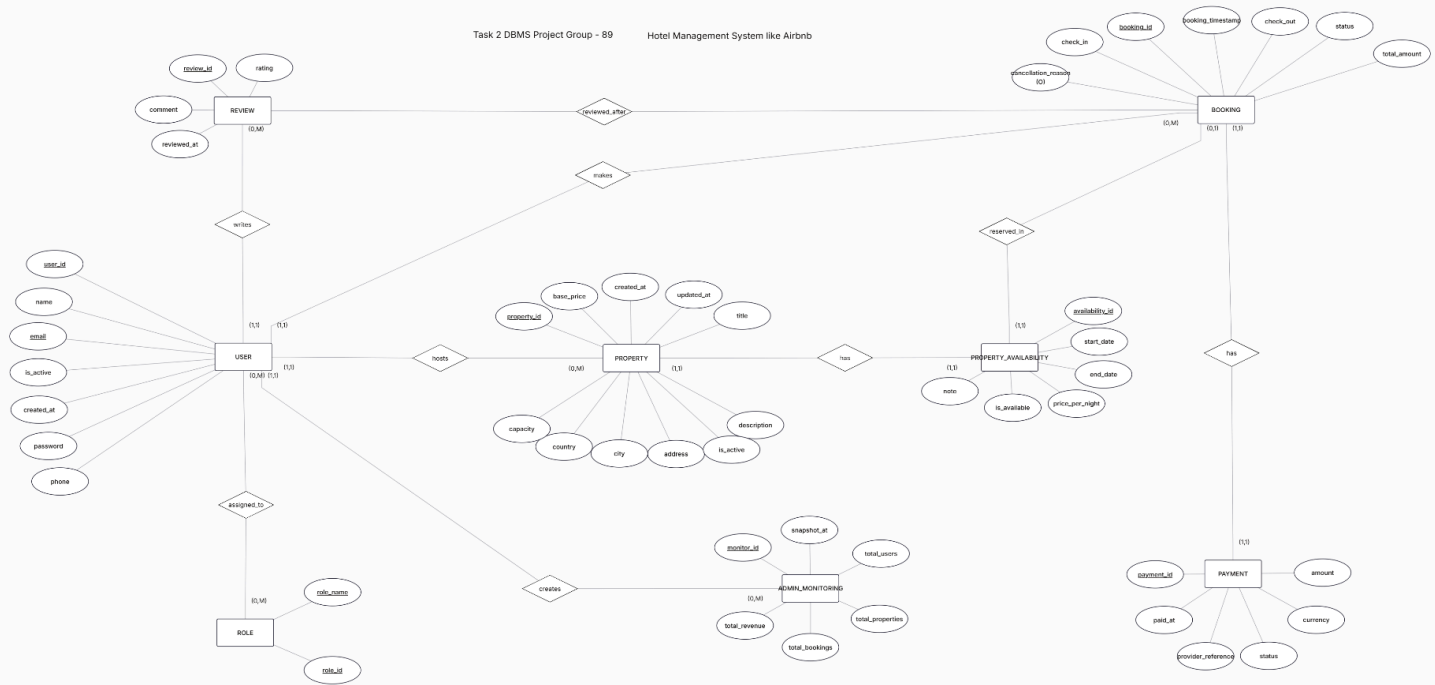
Task 2: ER Model and Relational Model Design

Airbnb-Style Accommodation Booking System – Group 89

1. Overview

After defining the scope, stakeholders, functional requirements, and business rules in Task 1, the next step in the database design process was to model the system using:

1. A Conceptual ER (Entity–Relationship) Model



2. A Relational Model derived from the ER model

The ER model captures the real-world entities involved in the system and the relationships between them, while the relational model converts this conceptual design into tables suitable for implementation in MySQL.

The design focuses on maintaining data integrity, preventing booking conflicts, and supporting the interactions between guests, hosts, and administrators.

2. Conceptual ER Model

The ER model represents the system at a high level using entities, attributes, and relationships with cardinalities.

2.1 Entities and Attributes

USER

Represents both guests and hosts using the system.

Attributes:

- user_id (Key)
 - name
 - email (Unique)
 - phone (Unique)
 - password
 - created_at
 - is_active
-

ROLE

Defines the role assigned to a user (guest, host, admin).

Attributes:

- role_id (Key)
 - role_name
-

PROPERTY

Represents the accommodation listed by a host.

Attributes:

- property_id (Key)

- title
- description
- address
- city
- country
- capacity
- base_price
- created_at
- updated_at
- is_active

PROPERTY_AVAILABILITY

Stores availability slots and dynamic pricing for properties.

Attributes:

- availability_id (Key)
- start_date
- end_date
- price_per_night
- is_available
- note

BOOKING

Represents a reservation made by a guest.

Attributes:

- booking_id (Key)
 - check_in
 - check_out
 - booking_timestamp
 - status
 - total_amount
 - cancellation_reason
-

PAYMENT

Stores payment details for bookings.

Attributes:

- payment_id (Key)
 - amount
 - currency
 - status
 - provider_reference
 - paid_at
-

REVIEW

Stores feedback given by guests after completing a stay.

Attributes:

- review_id (Key)
 - rating
 - comment
 - reviewed_at
-

ADMIN_MONITORING

Used by administrators to track system statistics.

Attributes:

- monitor_id (Key)
 - snapshot_at
 - total_users
 - total_properties
 - total_bookings
 - total_revenue
-

3. Relationships and Cardinalities in ER Model

The following relationships define how entities interact.

USER — hosts — PROPERTY (1 : N)

- One user (host) can list multiple properties.
- Each property is owned by exactly one user.

USER — makes — BOOKING (1 : N)

- A user (guest) can make multiple bookings.
- Each booking is made by one user.

PROPERTY — has — PROPERTY_AVAILABILITY (1 : N)

- One property can have many availability slots.
- Each availability slot belongs to one property.

PROPERTY_AVAILABILITY — reserved_in — BOOKING (1 : 1)

- Each availability slot can be booked once.
- Each booking corresponds to exactly one availability slot.

BOOKING — has — PAYMENT (1 : 1)

- Each booking has exactly one payment record.
- Each payment belongs to one booking.

BOOKING — reviewed_after — REVIEW (1 : 0..1)

- A booking may or may not have a review.
- Each review is linked to one booking.

USER — writes — REVIEW (1 : N)

- A user can write multiple reviews.
- Each review is written by one user.

USER — assigned_to — ROLE (M : N)

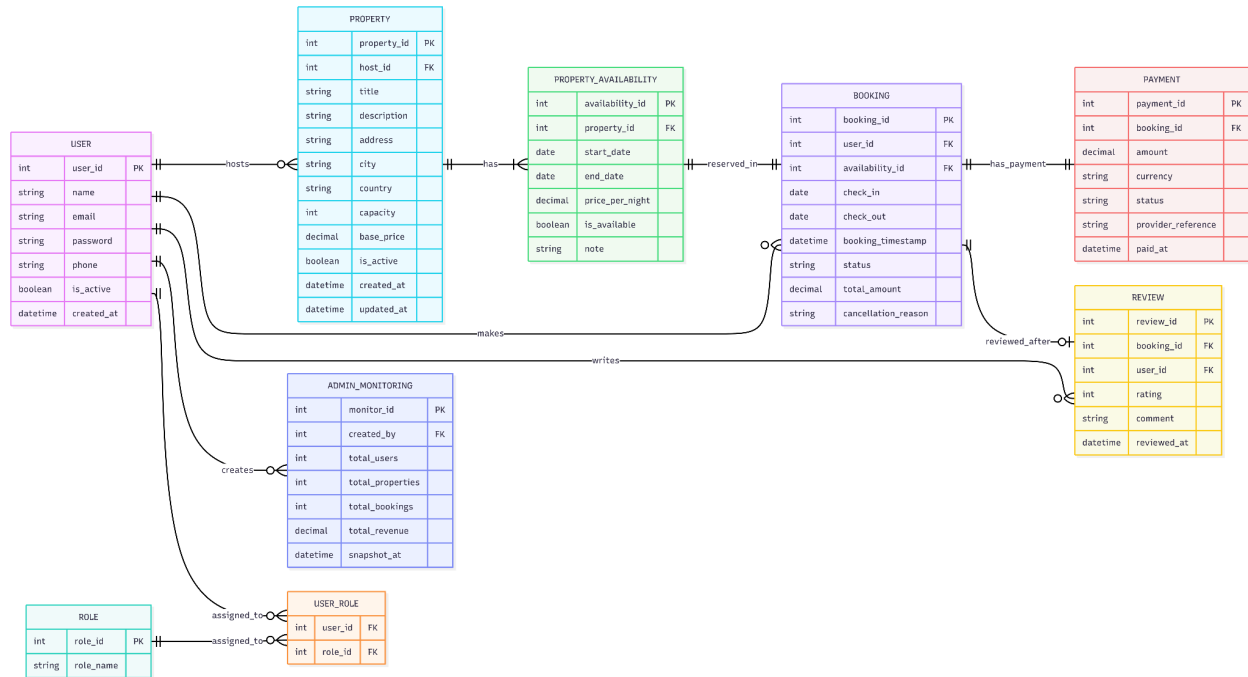
- A user can have multiple roles.
- A role can be assigned to multiple users.

This many-to-many relationship is important in the conversion to the relational model.

USER — creates — ADMIN_MONITORING (1 : N)

- An admin user can create multiple monitoring snapshots.
 - Each snapshot is created by one user.
-

4. Relational Model



The relational model is derived from the ER model by converting entities into tables and relationships into foreign keys.

4.1 Tables Created

The following tables are created:

- USER
- ROLE
- USER_ROLE
- PROPERTY
- PROPERTY_AVAILABILITY
- BOOKING
- PAYMENT

- REVIEW
 - ADMIN_MONITORING
-

4.2 Why USER_ROLE table exists

In the ER model, **USER** and **ROLE** are connected by a many-to-many relationship.

During ER to relational conversion, every many-to-many relationship is converted into a separate table.

Hence, the table:

USER_ROLE (user_id, role_id)

is created to resolve this relationship.

This is why the relational model has 9 tables while the ER model has 8 entities.

4.3 Use of Primary and Foreign Keys

Foreign keys are used to enforce business rules:

- PROPERTY.host_id → USER.user_id
- PROPERTY_AVAILABILITY.property_id → PROPERTY.property_id
- BOOKING.user_id → USER.user_id
- BOOKING.availability_id → PROPERTY_AVAILABILITY.availability_id
- PAYMENT.booking_id → BOOKING.booking_id
- REVIEW.booking_id → BOOKING.booking_id
- REVIEW.user_id → USER.user_id
- ADMIN_MONITORING.created_by → USER.user_id

These constraints ensure data consistency and prevent invalid records.

5. Design Justification

Some important design decisions:

- Availability is separated into a different entity to prevent double booking and allow flexible pricing.
- Reviews are linked to bookings to ensure only valid guests can review.
- Admin monitoring is modeled as an entity to support analytics.
- Role management is kept flexible using a many-to-many design.

6. Conclusion

The ER model captures the real-world structure of the Airbnb-style booking system using entities and relationships. The relational model systematically converts this design into tables with keys and constraints suitable for implementation.

This design ensures:

- Prevention of booking conflicts
- Proper user and role management
- Accurate tracking of bookings, payments, and reviews
- Support for administrative monitoring and analytics

The models together provide a strong foundation for implementing the database in MySQL while satisfying all functional and business requirements defined in Task 1.